데이터사이언스

Lecture 07 데이터 프레임

데이터 프레임

- 2차원 배열의 원소로 구성된 하나의 스프레드시트를 의미하며 표와 같다고 볼 수 있음
- 2차원 배열은 행과 열의 형태로 주어진 원소값을 저장하기 위한 프레임으로 엑셀에서 사용하는 시트(Sheet)와 같은 의미



그림 7-1 데이터 프레임 구조와 인덱스

인덱스

- 데이터 원소들을 효율적으로 그룹화하고 참조가 용이하도록 제공하는 구분자를 의미
- 인덱스는 데이터 프레임에서 원소의 값을 구분하기 위한 고유의 식별 번호를 사용하며 대학생에게 부여되는 학번과 같은 역할을 수행
- 서버에 저장되어 있는 데이터를 보호하기 위해서도 인덱스를 활용

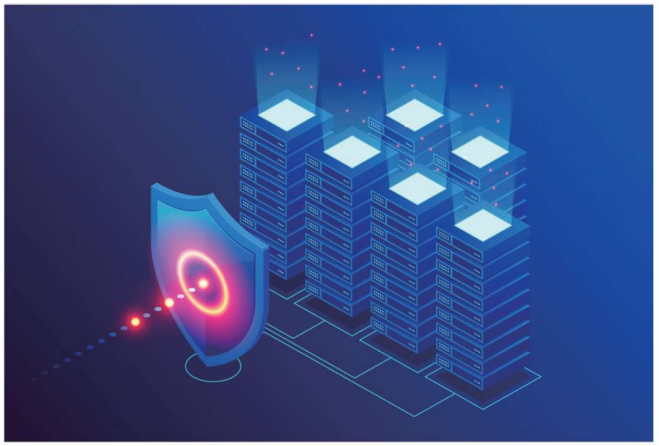


그림 7-2 델 EMC 글로벌 데이터 보호 인덱스 (출처 : 네이버)

생성 함수 – DataFrame()

- 데이터 프레임을 생성할 때는 판다스의 DataFrame( ) 함수를 사용하며 판다스란 데이터를 처리하기 위한 라이브러리를 의미
- import 명령으로 pandas 패키지를 불러와 객체를 생성하는 과정으로 데이터 프레임을 생성
- pandas 패키지는 아나콘다를 설치하게 되면 자동으로 설치되는 패키지이므로 따로 설치할 필요는 없음

```
import pandas as 객체명
변수명 = 객체명.DataFrame([[첫 번째 행의 배열 원소], [두 번째 행의 배열 원소]])
print(변수명)
```

데이터 프레임 생성하기

◦ 2차원 배열로 구성된 데이터 프레임을 생성하기 위해 다음 표와 같이 필요한 환자 코드, 연령, 성별, 거주지 데이터를 제시

표 7-1 환자 명세서

환자 코드	연령	성별	거주지
A00301	23	남	서울특별시
D00809	53	여	제주특별자치도

• **Step 01** | 환자 명세서에 대한 데이터 프레임을 생성하기 위해 다음과 같이 소스 코드를 작성합니다. 데이터 프레임 df는 임의로 생성한 변수명이므로 사용자가 다르게 선언해도 됩니다.

▶▶ Chap-07/Ex07-01.jpynb

```
01  import pandas as pd          # 판다스 객체 생성
02
03  df = pd.DataFrame([['환자 코드', '연령', '성별', '거주지'],
04                     ['A00301', 23, '남', '서울특별시'],
05                     ['D00809', 53, '여', '제주특별자치도']])
06  print(df)                    # 데이터 프레임 출력
```

행 인덱스와 열 이름 설정

- 방법1: DataFrame( ) 함수의 인자로 전달하는 방법
- 방법2: rename( ) 메소드를 사용하는 방법

열 이름(column name)

	0	1	2	3
0	환자 코드	연령	성별	거주지
1	A00301	25	남	서울특별시
2	D00809	19	여	제주특별자치도

행 인덱스 (row index)

그림 7-3 행 인덱스와 열 인덱스

DataFrame( ) 함수의 인자로 전달

- 2차원 배열을 DataFrame( ) 함수의 인자로 전달하여 직접 지정하려면 다음과 같이 선언

```
import pandas as 객체명
변수명 = 객체명.DataFrame([[첫 번째 인덱스를 제외한 배열 원소],
                           [두 번째 인덱스를 제외한 배열 원소]],
                          index = [인덱스로 사용할 행의 배열 원소명 선언],
                          columns = [인덱스 열을 제외한 배열 원소의 컬럼명 선언])
print(변수명)
```

DataFrame( ) 함수의 인자로 전달

- 주어진 데이터를 대상으로 데이터 프레임을 생성하고 변수명 df에 대입하여 행 인덱스는 df.index, 열 인덱스는 df.columns로 설정하면 다음 그림과 같이 표현하여 쉽게 이해할 수 있음



그림 7-4 행 인덱스와 열 이름 설정

DataFrame( ) 함수의 인자로 전달

예제 07 – 02 데이터 프레임의 행 인덱스와 열 이름 설정 후 각각 출력하기

- **Step 01** | 환자 명세서에 대한 데이터 프레임의 행 인덱스와 열 이름을 출력하기 위해 다음과 같이 소스 코드를 작성합니다.

▶▶ Chap-07/Ex07-02.jpynb

```
01  import pandas as pd
02
03  df = pd.DataFrame([[23, '남', '서울특별시'],
04                      [53, '여', '제주특별자치도']],
05                      index = ['A00301', 'D00809'],
06                      columns = ['연령', '성별', '거주지'])
07  print(df)           # 데이터 프레임 출력
08  print('\n')        # 한 줄 띄우기
09  print(df.index)     # 행 인덱스 출력
10  print(df.columns)   # 열 이름 출력
```

DataFrame( ) 함수의 인자로 전달

- 2차원 배열을 DataFrame( ) 함수의 인자로 전달하여 생성한 데이터 프레임 df를 사용하여 행 인덱스와 열 이름을 변경하는 사용형식:
  - 데이터 프레임 ⇒ df
  - 행 인덱스 변경 ⇒ df.index = 설정하려는 행 인덱스 배열
  - 열 이름 변경 ⇒ df.columns = 설정하려는 열 이름 배열

rename( ) 메소드

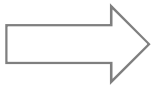
- rename( ) 메소드를 사용하여 데이터 프레임에 존재하는 행 인덱스와 열 이름의 일부를 선택적으로 변경할 수 있음
- 이 방법은 원본 객체는 그대로 두고 새로운 데이터 프레임 객체를 생성하여 사용하는 방법
- 만약 원본 객체를 변경하려면 inplace=True 옵션을 사용해야 함
- 데이터 프레임 df와 rename( ) 메소드를 사용하여 행 인덱스와 열 이름을 변경하기 위한 사용형식은 다음과 같음

```
• 데이터 프레임 ⇒ df
• 행 인덱스 변경 ⇒ df.rename(index={'기존 인덱스':'새로운 인덱스', ...})
• 열 이름 변경 ⇒ df.rename(columns={'기존 열 이름':'새로운 열 이름', ...})
```

rename( ) 메소드

- rename()을 이용하여 앞의 데이터프레임에서 환자코드를 '환자1', '환자2'로 변경하고 '연령'은 age, '성별'은 'gender', '거주지'는 'addr'로 변경해보자.

```
        연령 성별       거주지
A00301  23   남     서울특별시
D00809  53   여   제주특별자치도
```



```
       age gender       addr
환자1   23      남     서울특별시
환자2   53      여   제주특별자치도
```

행 데이터 선택

◦ 특정 데이터를 선택하는 2가지 방법

- loc 인덱서 사용 : 인덱스의 이름(특정 레이블)을 기준으로 선택
- iloc 인덱서 사용 : 특정 위치를 인덱스의 정수값으로 지정하여 선택

표 7-2 loc와 iloc 인덱서 비교

구분	loc	iloc
검색 대상	인덱스에서 특정 레이블이 있는 행	인덱스의 특정 위치에 있는 행
대상 범위	검색 범위의 마지막도 포함	검색 범위의 마지막은 불포함(정수만 사용 가능)
사용 예시	['e':'g'] ⇒ 'e', 'f', 'g'	[4:9] ⇒ 4, 5, 6, 7, 8

여기서 잠깐! 인덱서에 대한 이해

데이터 프레임에 존재하는 데이터 중에서 행 데이터를 선택하려면 먼저 인덱서에 대한 이해가 필요합니다. 인덱서(Indexer)란 입력된 문서에 대해서 색인을 자동으로 작성해 주는 프로그램을 의미합니다.

행 데이터 선택

표 7-3 건강관리 대상자의 생체 데이터

성명	나이	성별	키	체중
이겨레	25	남	185	89
조약돌	19	여	163	53

데이터 프레임 : df

	성명	나이	성별	키	체중
0	이겨레	25	남	185	89
1	조약돌	19	여	163	53

df.loc['이겨레'] 또는 df.iloc[0]

0	이겨레	25	남	185	89

df.loc[['이겨레', '조약돌']] 또는 df.iloc[[0,1]]

0	이겨레	25	남	185	89
1	조약돌	19	여	163	53

df.loc['이겨레':'조약돌'] 또는 df.iloc[0:2]

0	이겨레	25	남	185	89
1	조약돌	19	여	163	53

그림 7-5 지정한 특정 행 데이터만 선택하여 출력

행 데이터 선택

- 데이터 프레임에서 지정하는 특정 행의 데이터만 선택하여 출력하려면 다음과 같이 선언

  - 데이터 프레임 ⇒ df
  - loc 인덱서 ⇒ 변수명 = df.loc['인덱스 이름']
  - iloc 인덱서 ⇒ 변수명 = df.iloc[정수형 인덱스]

행 데이터 선택

예제 07 – 04 데이터 프레임에서 인덱서로 각 행의 데이터를 선택하여 출력하기

- **Step 01** | 데이터 프레임을 생성하고 loc 인덱서와 iloc 인덱서를 사용하여 인덱스를 지정한 특정 행의 데이터를 선택하여 출력하기 위해 다음과 같이 소스 코드를 작성합니다.

▶▶ Chap-07/Ex07-04.jpynb

```
01  import pandas as pd
02
03  bio_data = {'나이':[25, 19], '성별':['남', '여'],
04              '키':[185, 163], '체중':[89, 53]}
05
06  # DataFrame( ) 함수를 사용하고 성명을 인덱스로 설정하여 변수 df에 저장
07  df = pd.DataFrame(bio_data, index=['이겨레', '조약돌'])
08
09  # 데이터 프레임 df 출력
10  print(df)
11  print('\n')
12
```

'이겨레' 행은 loc 인덱서, '조약돌' 행은 iloc 인덱서를 사용하여 행을 출력해보자.

행 데이터 선택

- 행 인덱스에 대해 리스트 형태로 2개 이상을 선언하게 되면 여러 개의 행에 대한 데이터를 동시에 선택할 수 있음
- 여기서 주의해야 할 사항은 인덱스를 설정할 때 대괄호([ ]) 2개를 이중으로 사용해야 하며 그렇지 않으면 문법 오류가 발생하게 됨

- 데이터 프레임 ⇒ df
- loc 인덱서 ⇒ 변수명 = df.loc[['인덱스 이름1', '인덱스 이름2', ...]]
- iloc 인덱서 ⇒ 변수명 = df.iloc[[0, 1, ...]]

- '이겨례'와 '조약돌'을 행을 loc 인덱서와 iloc 인덱서를 사용하여 출력해보자.

행 데이터 선택

- 인덱스의 범위에 대해 콜론(:)을 사용하여 설정하게 되면 여러 개의 행 데이터를 한꺼번에 선택할 수 있으며 인덱스 범위를 설정하는 사용형식은 다음과 같이 선언

- 데이터 프레임 ⇒ df
- loc 인덱서 ⇒ 변수명 = df.loc['인덱스의 시작 이름':'인덱스의 마지막 이름']
- iloc 인덱서 ⇒ 변수명 = df.iloc[시작 인덱스:끝 인덱스+1]

- 인덱스 범위를 설정할 때 주의해야 할 사항은 iloc 인덱서의 경우 범위 설정의 마지막 인덱스는 포함시키지 않기 때문에 끝 인덱스에 1을 더해 준 값을 마지막 범위로 선언해 주어야 함
- '이겨례'와 '조약돌'을 행을 슬라이싱 기법을 사용하여 출력해보자.

퀴즈1: 2행 간격으로 슬라이싱?

퀴즈2: 역순으로 정렬하는 범위 슬라이싱?

행 데이터 선택

도전 07 – 01 다음 지시사항을 수행하여 파이썬 소스 코드를 작성하시오.

1. 아래와 같이 주어진 데이터를 보고 데이터 프레임 생성하고 데이터 프레임 출력하기

	컴퓨터	모니터	노트북	스마트폰
서울지점	869	374	569	197
부산지점	578	286	427	89
제주지점	321	148	335	66

2. DataFrame( ) 함수를 사용하고 지점명을 인덱스로 설정하여 변수 df에 저장하기
3. loc 인덱서를 사용하여 서울지점의 행 데이터만 선택하여 출력하기
4. iloc 인덱서를 사용하여 부산지점의 행 데이터만 선택하여 출력하기

열 데이터 선택

- 첫 번째 방법 : 열 이름을 따옴표와 함께 대괄호([ ]) 안에 선언
- 두 번째 방법 : 문자열인 경우만 사용하는 방법으로 도트(.) 다음에 열이름을 입력하는 방법

- 데이터 프레임 ⇒ df
- 대괄호 사용 ⇒ 변수명 = df['열 이름']
- 도트(.) 사용 ⇒ 변수명 = df.열 이름

열 데이터 선택

- 데이터 프레임에서 '성명' 열 데이터와 '나이' 열 데이터만 각각 따로 선택하여 출력되는 과정을 다음 그림과 같이 표현

데이터 프레임 : df

	성명	나이	성별	키	체중
0	이겨레	25	남	185	89
1	조약돌	19	여	163	53

df['성명']

	성명
0	이겨레
1	조약돌

df.나이

	나이
0	25
1	19

그림 7-6 지정한 특정 열 데이터만 선택하여 출력

열 데이터 선택

- 데이터 프레임에서 3개(성명, 키, 체중)의 특정 열 데이터만 선택하여 출력하는 과정을 다음 그림과 같이 표현

데이터 프레임 : df

	성명	나이	성별	키	체중
0	이겨레	25	남	185	89
1	조약돌	19	여	163	53

df[['성명', '키', '체중']]

	성명	키	체중
0	이겨레	185	89
1	조약돌	163	53

df[['나이']]

	나이
0	25
1	19

그림 7-7 2개 이상의 열 데이터 선택과 데이터 프레임 타입 반환 형식

열 데이터 선택

도전 07 – 02 다음 지시사항을 수행하여 파이썬 소스 코드를 작성하시오.

1. 아래와 같이 주어진 데이터를 보고 데이터 프레임 생성하고 데이터 프레임 출력하기

	컴퓨터	모니터	노트북	스마트폰
서울지점	869	374	569	197
부산지점	578	286	427	89
제주지점	321	148	335	66

2. 도트(.)를 사용하여 지점명에 대한 1개의 열 데이터만 선택하여 출력하기
3. 대괄호([])를 사용하여 지점명과 노트북에 대한 2개의 열 데이터만 선택하여 출력하기
4. 선택된 데이터의 타입을 출력하기 위해 type( ) 메소드를 사용하기

7주차 과제#1

Eclass에 7주차 과제#1를 푸세요.

파이썬 함수

- **pd.read_csv()** — 데이터 불러오기
  CSV 파일을 읽어서 DataFrame으로 변환

```
df = pd.read_csv('파일명.csv')
```

- **df.shape** — 데이터 크기 확인
  DataFrame의 행 수와 열 수를 튜플로 반환

```
df.shape     # (n, m)
df.shape[0]  # n ← 행 수
df.shape[1]  # m ← 열 수
```

- **df.head()** / df.tail() — 데이터 미리보기

```
df.head()    # 처음 5개 행
df.head(10)  # 처음 10개 행
```

파이썬 함수

- **df['열이름'].unique()** — 중복 없는 목록
  해당 열에 어떤 값들이 있는지 중복 없이 보여줌

```
df['카테고리'].unique()
```

- **df.loc[]** — 이름으로 행/열 선택
  조건에 맞는 행만 골라낼 때 사용

```
# 조건으로 행 선택
df.loc[df['카테고리'] == '식품']

# 조건 두 개 동시에 (AND)
df.loc[(df['지점'] == '홍대점') & (df['카테고리'] == '음료')]
```

파이썬 함수

- **df.iloc[]** — 숫자로 행/열 선택
  인덱스 번호로 행을 선택할 때 사용

```
df.iloc[0]     # 첫 번째 행
df.iloc[2]     # 세 번째 행
df.iloc[0:5]   # 0~4번째 행 (5 미포함)
df.iloc[-5:]   # 마지막 5개 행
```

- **df['새열'] = 계산식** — 새 열 추가
  기존 열을 계산해서 새로운 열을 만듬

```
df[‘C'] = df[‘A'] * df[‘B']
```

파이썬 함수

- **list()** — 리스트로 변환

```
A_list = list(df['A'].unique())
```

- **sum() — 합계**
  리스트나 딕셔너리의 값들을 모두 더함

```
grand_total = sum(category_sales.values())
```

- **dict.items()** — 딕셔너리 순회
  딕셔너리를 for문으로 돌릴 때 키와 값을 동시에 꺼낼 수 있음

```
sum(category.values())          # 값들의 합계
max(qty.values())               # 값들 중 최댓값
list(qty.values()).index(max()) # 최댓값의 위치
```

파이썬 함수

- **sorted()** — 정렬

```
# 딕셔너리를 값 기준으로 내림차순 정렬
sorted_branch = dict(sorted(branch_sales.items(),
                            key=lambda x: x[1],
                            reverse=True))
```

- **format()** — 출력 형식 지정

```
format(2009, ',')   # "2,009" ← 천 단위 콤마
format(df.shape[0])  # “nrow"
"{}위: {}".format(A, B)  # 문자열 안에 변수 삽입
```

7주차 과제#2

"week7_market.csv" 데이터를 불러와 편의점 매출 데이터 분석을 하는 과제#2를 푸세요.